```
/-!
# A closed timelike curve as an *untrusted prover* in front of Lean's *trusted kernel*

Core Lean 4 only (no Mathlib); checked with Lean 4.35.0. Two small, machine-checked idea
s.

**1. `CTC` : the fixed-point trick.**
A classical CTC computer is a map `step` on the loop state; a history is *consistent*
iff what comes out of the loop equals what went in (`step x = x`).
Let the loop state be "candidate proof number `x`". The gadget keeps the candidate if th
e
checker accepts it, and otherwise passes the *next* candidate around the loop.
Then every consistent history is a valid proof, and a consistent history exists iff a va
lid
proof exists.  If none exists there is no deterministic consistent history (grandfather
paradox).  Deutsch's model repairs this with a *mixed* fixed point: the loop is then a
permutation of the candidates, so the uniform distribution is invariant (`step_lt`,
`step_injective_of_no_witness`), the device returns a random candidate, and the kernel
rejects it.

**2. `LCF` : the trust boundary.**
Whatever sits behind the `Prover` interface (a CTC, a halting oracle, an LLM, a bug),
a result that passes the kernel is true.  A hypercomputer can change *completeness and
speed*, never *soundness*.  `ctcProver` models the device with `Classical.choose`:
the CTC is, physically, what would make that non-computable selection computable.
-/

namespace CTC

/-- One pass around the loop for witness search over candidates `0 … n-1`.
`ok x` means "run Lean's kernel on candidate number `x` and see whether it accepts". -/
def step (n : Nat) (ok : Nat → Bool) (x : Nat) : Nat :=
  if ok x then x else (x + 1) % n

/-- A history is consistent iff what exits the loop equals what entered it. -/
def Consistent (n : Nat) (ok : Nat → Bool) (x : Nat) : Prop :=
  x < n ∧ step n ok x = x

/-- A valid candidate is always a consistent history. -/
theorem witness_consistent (n : Nat) (ok : Nat → Bool) (x : Nat)
    (hx : x < n) (h : ok x = true) : Consistent n ok x := by
  refine ⟨hx, ?_⟩
  simp [step, h]
```

```
/-- Soundness of the trick: every consistent history is a valid witness. -/
theorem consistent_is_witness (n : Nat) (hn : 2 ≤ n) (ok : Nat → Bool) (x : Nat)
    (hc : Consistent n ok x) : ok x = true := by
  obtain ⟨hx, hfix⟩ := hc
  cases hok : ok x with
  | true => rfl
  | false =>
    exfalso
    simp [step, hok] at hfix
    by_cases hlt : x + 1 < n
    · rw [Nat.mod_eq_of_lt hlt] at hfix
      omega
    · have hxn : x + 1 = n := by omega
      rw [hxn, Nat.mod_self] at hfix
      omega

/-- A consistent history exists exactly when some candidate below `n` is valid. -/
theorem consistent_exists_iff (n : Nat) (hn : 2 ≤ n) (ok : Nat → Bool) :
    (∃ x, Consistent n ok x) ↔ ∃ x, x < n ∧ ok x = true := by
  constructor
  · rintro ⟨x, hc⟩
    exact ⟨x, hc.1, consistent_is_witness n hn ok x hc⟩
  · rintro ⟨x, hx, h⟩
    exact ⟨x, witness_consistent n ok x hx h⟩

/-- Grandfather paradox: with no valid witness, the deterministic loop has no consistent
history at all. -/
theorem no_witness_no_history (n : Nat) (hn : 2 ≤ n) (ok : Nat → Bool)
    (hnone : ∀ x, x < n → ok x = false) : ¬ ∃ x, Consistent n ok x := by
  rw [consistent_exists_iff n hn ok]
  rintro ⟨x, hx, h⟩
  have h0 := hnone x hx
  simp [h0] at h

/-- The loop never leaves the candidate set. -/
theorem step_lt (n : Nat) (hn : 0 < n) (ok : Nat → Bool) (x : Nat) (hx : x < n) :
    step n ok x < n := by
  unfold step
  split
  · exact hx
  · exact Nat.mod_lt _ hn

```

```
/-- With no valid witness the loop is the cyclic shift `x ↦ x+1 mod n`, which is injecti
ve
on `{0,…,n-1}`; hence a permutation of that finite set, hence the uniform distribution i
s
invariant.  That uniform mixture is Deutsch's consistent state in the "no proof" case.
-/
theorem step_injective_of_no_witness (n : Nat) (ok : Nat → Bool)
    (hnone : ∀ x, x < n → ok x = false) (x y : Nat) (hx : x < n) (hy : y < n)
    (h : step n ok x = step n ok y) : x = y := by
  have key : ∀ z, z < n → step n ok z = if z + 1 < n then z + 1 else 0 := by
    intro z hz
    have hz0 := hnone z hz
    by_cases hlt : z + 1 < n
    · simp [step, hz0, hlt, Nat.mod_eq_of_lt hlt]
    · have hzn : z + 1 = n := by omega
      simp [step, hz0, hzn]
  rw [key x hx, key y hy] at h
  split at h <;> split at h <;> omega

end CTC

namespace LCF

/-- A formal system with a trusted, decidable proof checker (think: Lean's kernel). -/
structure FormalSystem where
  Stmt  : Type
  Proof : Type
  check : Proof → Stmt → Bool
  Holds : Stmt → Prop
  sound : ∀ p s, check p s = true → Holds s

/-- An untrusted prover: a CTC device, a halting oracle, an LLM, a bug. Anything. -/
abbrev Prover (S : FormalSystem) := S.Stmt → Option S.Proof

/-- Run the prover, but only believe what the kernel certifies. -/
def certify (S : FormalSystem) (P : Prover S) (s : S.Stmt) : Option S.Proof :=
  match P s with
  | none   => none
  | some p => if S.check p s then some p else none

/-- Soundness does not depend on the prover at all. -/
theorem certify_sound (S : FormalSystem) (P : Prover S) (s : S.Stmt) (p : S.Proof)
    (h : certify S P s = some p) : S.Holds s := by
```

```
  unfold certify at h
  cases hP : P s with
  | none => simp [hP] at h
  | some q =>
    simp only [hP] at h
    by_cases hc : S.check q s = true
    · exact S.sound q s hc
    · simp [hc] at h

/-- The prover only buys *completeness*: if it can hand back a checkable proof whenever one
exists, so does the certified pipeline. -/
theorem certify_complete (S : FormalSystem) (P : Prover S)
    (hP : ∀ s p, S.check p s = true → ∃ q, P s = some q ∧ S.check q s = true)
    (s : S.Stmt) (p : S.Proof) (hp : S.check p s = true) :
    ∃ q, certify S P s = some q := by
  obtain ⟨q, hq, hc⟩ := hP s p hp
  exact ⟨q, by simp [certify, hq, hc]⟩

open Classical in
/-- The CTC device as a prover: it returns the contents of *a* consistent history of the
search loop, if one exists.  In Lean this selection is `Classical.choose`; a physical CTC
would be what makes it computable.  `enc i` is the `i`-th candidate proof object. -/
noncomputable def ctcProver (S : FormalSystem) (n : Nat) (enc : Nat → S.Proof) : Prover S :=
  fun s =>
    if h : ∃ x, CTC.Consistent n (fun i => S.check (enc i) s) x then
      some (enc (Classical.choose h))
    else none

/-- Completeness for every proof among the first `n` candidates. -/
theorem ctcProver_complete (S : FormalSystem) (n : Nat) (hn : 2 ≤ n) (enc : Nat → S.Proof)
    (s : S.Stmt) (i : Nat) (hi : i < n) (hproof : S.check (enc i) s = true) :
    ∃ q, ctcProver S n enc s = some q ∧ S.check q s = true := by
  have hex : ∃ x, CTC.Consistent n (fun j => S.check (enc j) s) x :=
    ⟨i, CTC.witness_consistent n _ i hi hproof⟩
  refine ⟨enc (Classical.choose hex), ?_, ?_⟩
  · simp [ctcProver, hex]
  · exact CTC.consistent_is_witness n hn _ _ (Classical.choose_spec hex)

/-- End to end: anything the kernel accepts from the CTC device is true. -/
```

```
theorem ctc_pipeline_sound (S : FormalSystem) (n : Nat) (enc : Nat → S.Proof)
    (s : S.Stmt) (p : S.Proof) (h : certify S (ctcProver S n enc) s = some p) :
    S.Holds s :=
  certify_sound S _ s p h

end LCF

/-! ### Concrete toy run
Candidates `0…19`, checker "x² = 49".  Candidate 7 is a consistent history; the same loop
with checker "x² = 50" (no solution) moves candidate 7 along, so 7 is not consistent. -/
example : CTC.step 20 (fun x => x * x == 49) 7 = 7 := by decide
example : CTC.step 20 (fun x => x * x == 50) 7 ≠ 7 := by decide

/-! ### Where the trust sits
The trust-boundary theorems (`certify_sound`, `consistent_is_witness`) use no choice axiom.
Only the *device model* `ctcProver_complete` does: `Classical.choice` is the oracle.
Anything beyond Lean's kernel that you believe about a real CTC would show up here, as an
explicit hypothesis, never inside the checked proof. -/
#print axioms CTC.consistent_is_witness
#print axioms LCF.certify_sound
#print axioms LCF.ctcProver_complete

```